



Dogecoin Price Prediction with Machine Learning

Dilawer Khan



Contents

1.Introduction.....	3
1.1 Context	3
1.2 Problem	3
1.3 Presentation of the Problem.....	3
2. Implementation.....	3
2.1 Data Collection.....	3
2.1.1 Loading and Understanding the Dataset	3
2.1.2 Dataset Information	4
2.1.3 Missing Values	4
2.1.4 Summary Statistics.....	4
2.1.5 Visualization.....	4
2.2 Data Preprocessing	5
2.2.1 Missing Value Handling	5
2.3 Feature Engineering.....	5
2.3.1 Moving Averages	5
2.3.2 Rate of Return (Daily Returns)	5
2.3.3 Volatility (7-day Rolling Standard Deviation)	5
2.3.4 Lag Feature (Closing Price of the Previous Day)	5
2.4 Data Visualization	5
2.4.1 Closing Price with Moving Averages	6
2.4.2 Daily Returns	6
2.4.3 7-Day Rolling Volatility.....	7
2.5 Feature Selection	7
2.5.1 Correlation Analysis.....	7
2.5.2 Feature Selection Decision	8
2.5.3 Final Features for Prediction	9
2.6 Model Selection, Justification and Explanation.....	9
2.6.1 Model Configuration.....	9

2.6.2 Linear Regression	9
2.6.3 Random Forest Regressor.....	11
2.6.4 XGBoost Regressor	12
2.6.5 ARIMA Model	13
2.6.6 LSTM Model.....	16
2.6.7 Comparative Analysis.....	18
2.7 Training Parameters.....	18
2.7.1 Hyperparameter Tuning.....	18
2.7.2 Performance Metrics.....	18
3. Tools and Technologies Used.....	19
4. Execution Details	19
4.1 Different Executions	19
4.1.1 Step-by-Step Execution	19
4.1.2 Execution for Model Training	19
4.1.3 Execution for Model Evaluation	19
4.2 Final Execution	19
4.2.1 How to Run the Final Version	19
4.2.2 Expected Outputs	19
5. Results	20
5.1 Performance Analysis.....	20
6. Conclusion	20
6.1 Summary and Findings	20
6.2 Future Work and Improvements	21

1.Introduction

1.1 Context

Cryptocurrencies have gained much popularity in recent times, and one of the most popular digital currencies is Dogecoin. It is hard to forecast the price of cryptocurrencies because they are very volatile and sensitive to market trends.

1.2 Problem

In this project, we will develop different machine learning models to predict the price of Dogecoin based on given historical data. Our aim is to analyze past price movements, process financial data, and use machine learning algorithms for prediction of future price movements based on given historical data. In doing so, we can also see that how machine learning can be used in financial forecasting.

1.3 Presentation of the Problem

This report details the process of data collection, preprocessing, model selection, and evaluation to build an effective Dogecoin price forecasting model for prediction of prices. The methods applied in this project will also aid in learning time series forecasting and its applications in cryptocurrency price forecasting.

2. Implementation

2.1 Data Collection

We require historical price data to create a machine learning model for price prediction of Dogecoin. We utilized a historical dataset of Dogecoin prices, which includes the following columns:

- Date – The date of the record
- Open – The price at which Dogecoin opened on that day
- High – The price on that day when Dogecoin was at its peak
- Low – The price on that day when Dogecoin hit its lowest
- Close – The price at which Dogecoin closed on that day
- Adj Close – The adjusted closing price, considering splits and dividends
- Volume – The volume of Dogecoins traded on that day

2.1.1 Loading and Understanding the Dataset

We loaded the dataset into a pandas DataFrame and made a few initial checks:

- Shown the first few rows to gain an understanding of the structure of the dataset
- Checked for missing values in the dataset
- Made summary statistics to view the distribution of data
- Visualized the closing price over time to gain an understanding of the trend

2.1.2 Dataset Information

The dataset comprises 1,761 rows and 7 columns, but has one missing value in each column.

2.1.3 Missing Values

There was one missing value in the Open, High, Low, Close, Adj Close, and Volume columns.

2.1.4 Summary Statistics

- The minimum closing price was \$0.001038, and the maximum closing price was \$0.684777.
- The average closing price was approximately \$0.0596, which indicates that Dogecoin has experienced extreme price fluctuations over time.
- The dataset includes a time span from November 9, 2017, to September 4, 2022.

2.1.5 Visualization

We visualized the closing price over time, which indicates evident fluctuations in Dogecoin's price, which indicates trends and volatility in the market.



Figure 1 Dogde Coin Closing Over Time

2.2 Data Preprocessing

We cleaned and preprocessed the data prior to training our machine learning model. This is to ensure that our model can learn efficiently from patterns in data.

2.2.1 Missing Value Handling

We had some missing values for a few columns in our dataset. For the sake of data consistency, we removed rows with missing values. After this, our dataset was left with 1,760 rows and no missing values.

2.3 Feature Engineering

We derived extra features from the provided dataset to enhance the predictive capability of our model.

2.3.1 Moving Averages

- 7-day Moving Average (MA7): The 7-day average closing price.
- 21-day Moving Average (MA21): The 21-day average closing price.

These features smooth out price volatility and detect trends.

2.3.2 Rate of Return (Daily Returns)

- Return_1d: The daily percentage change in closing price.

This aids in learning price momentum and volatility.

2.3.3 Volatility (7-day Rolling Standard Deviation)

- Volatility_7d: The 7-day rolling standard deviation of daily returns.

This captures short-run price volatility and market risk.

2.3.4 Lag Feature (Closing Price of the Previous Day)

- Lag_1: The closing price of the previous day.

This feature aids the model in learning from past price behavior.

We removed any extra missing values introduced during rolling calculations after feature engineering.

2.4 Data Visualization

To better understand the dataset, we plotted the following:

2.4.1 Closing Price with Moving Averages

A plot of the Dogecoin closing price along with the 7-day and 21-day moving averages, which aid in detecting trends.

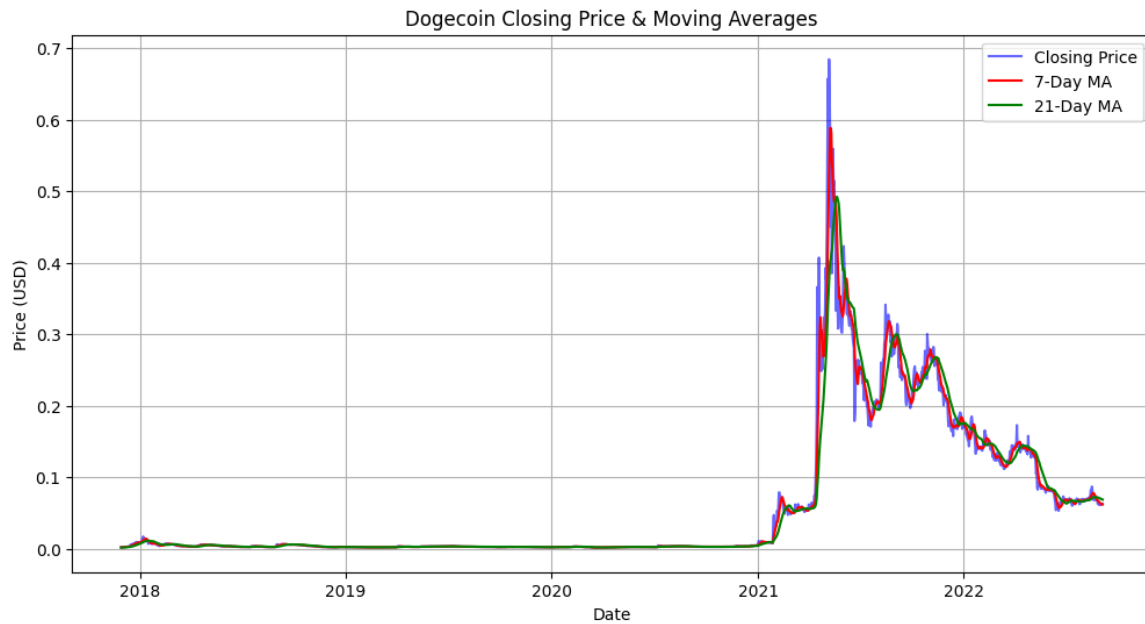


Figure 2 Doge Coin Prices and Moving Average

2.4.2 Daily Returns

A plot of the percentage change in price on each day, indicating the volatility of Dogecoin.

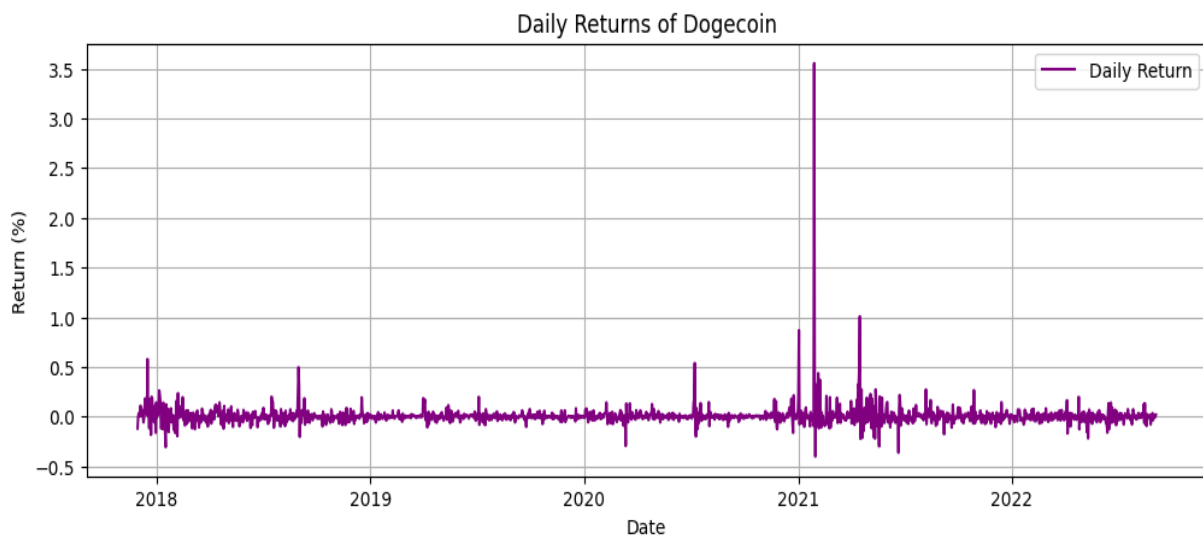


Figure 3 Daily Returns of Doge Coin

2.4.3 7-Day Rolling Volatility

A plot of price volatility fluctuation over time.

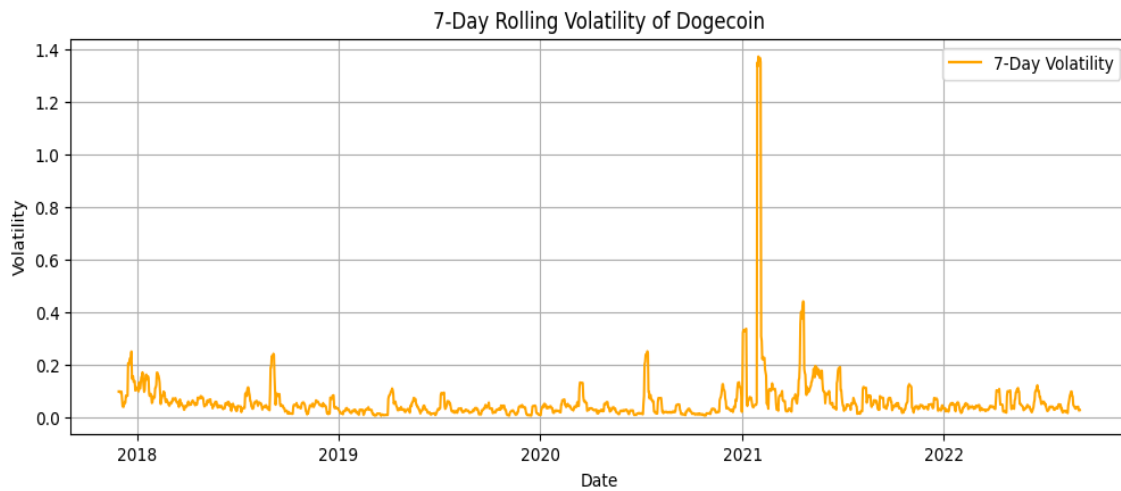


Figure 4 7-day Rolling Volatility of Dodge Coin

The graphical illustrations presented informative perspectives on the price fluctuations of Dogecoin in the past and assisted in determining appropriate features for prediction.

2.5 Feature Selection

To develop an efficient machine learning model, we need to select features that are strongly predictive of Dogecoin's closing price. One of the methods is to analyze the correlation of different features with the closing price.

2.5.1 Correlation Analysis

We calculated the correlation between the closing price and engineered features. Correlation heatmap was utilized to plot the correlations. The correlation values are as follows:

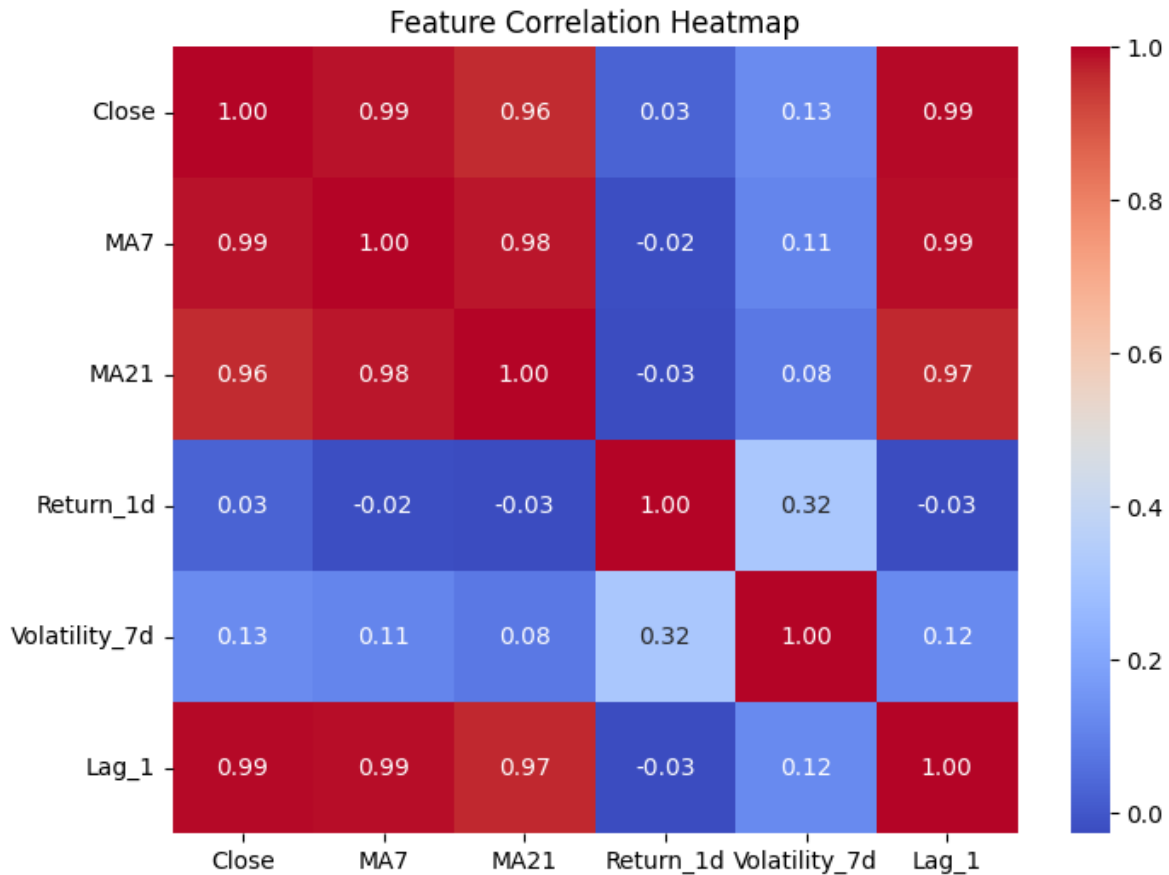


Figure 5 Corelation HeatMap

2.5.1.1 Feature Correlation with Closing Price

- MA7 (7-Day Moving Average) :0.987
- MA21 (21-Day Moving Average) : 0.962
- Return_1d (Daily Returns) : 0.026
- Volatility_7d (7-Day Volatility) : 0.128
- Lag_1 (Previous Day's Close) : 0.992

2.5.2 Feature Selection Decision

According to the correlation values:

2.5.2.1 Highly Correlated Features

- Lag_1 (0.992) and MA7 (0.987) are most correlated with the closing price.
- MA21 (0.962) is also highly correlated.

These features will be included in the model because they are helpful for trend information.

2.5.2.2 Weakly Correlated Features

- Return_1d (0.026) has almost zero correlation with the closing price.
- Volatility_7d (0.128) has weak correlation.

These features are less helpful for direct price prediction but may be helpful for risk assessment models.

2.5.3 Final Features for Prediction

Finally following features were selected for inputs:

- Lag_1 (Previous Day's Close)
- MA7 (7-Day Moving Average)
- MA21 (21-Day Moving Average)

These features will help us to capture the past price trends and make our predictions more precise for current time.

2.6 Model Selection, Justification and Explanation

To derive the closing price for Dogecoin, we constructed different models that uses historical price trends. These models relate to earlier data patterns and helps to derive future prices.

2.6.1 Model Configuration

Our choice of features for the prediction consisted of three very important branches:

- Lag_1: The previous day's close price.
- MA7: A 7-day moving average, allowing it to factor in a range of medium term fluctuations.
- MA21: A 21-day moving average, which is good for identifying longer trends.
- Target Variable: Close Price

The data was divided in the following manner:

- 80% Training Data-Used to build the model.
- 20% Testing Data-Used to evaluate the model performance on unseen data.

2.6.2 Linear Regression

We used a Linear Regression algorithm as it is one of the simplest yet quite powerful algorithms in finding relationships between past and future prices. The model was built using the training data and was kept in a test with remaining 20% for the evaluation of prediction performance.

The evaluation of our model's performance was made based on three key metrics:

Metric	Value
Mean Absolute Error	0.0047
RMSE	0.0074
R ² Score	0.9862

MAE (0.0047): The programmed model predicted the closing price off by 0.0047 units on average from the actual observed value.

RMSE (0.0074): Relative error indicators

R² Score (0.9862): The model accounts for 98.6% of price variations and is therefore a fair amount trustworthy.

In order to depict the accuracy of the model we generated a graph that illustrates actual vs. predicted closing prices:

The black dotted line signifies the actual market prices, while the blue line shows the predictions made through the model.

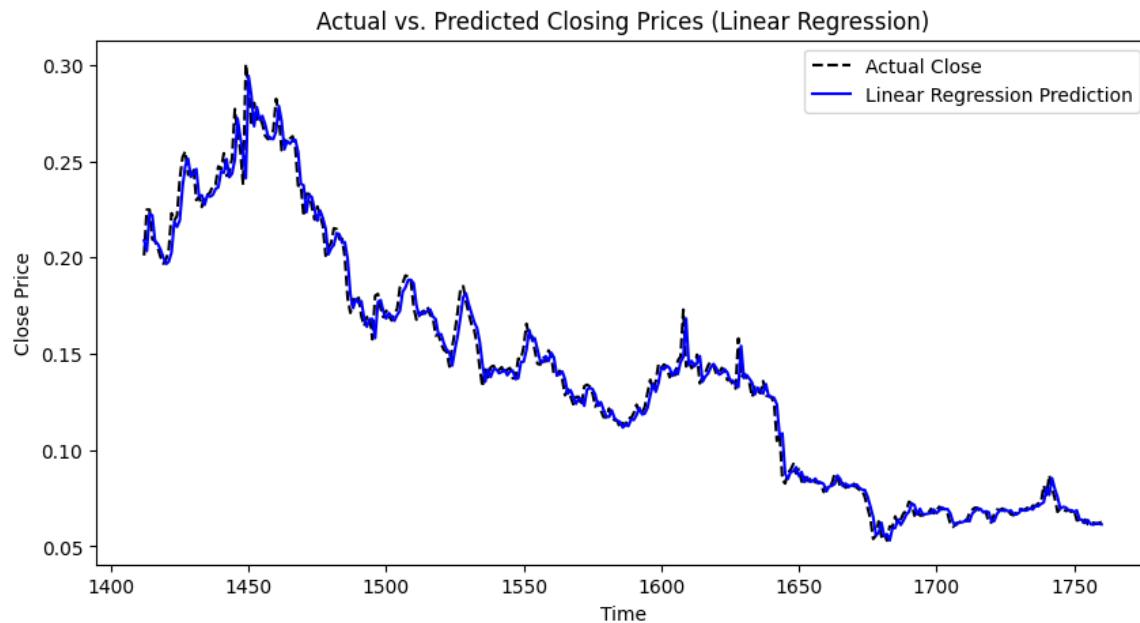


Figure 6 Linear Regression Predictions

The close fit of the two lines is indicative that this model mentions market trends and gives valid forecasts.

2.6.3 Random Forest Regressor

We built a Random Forest Regressor, an ensemble learning approach that constructs many decision trees and then averages the predictions so as to increase accuracy and avoid overfitting. It was trained on historical stock price, which contained 80% data for training and the remaining 20% for testing.

The evaluation of our model's performance was made based on three key metrics:

Metric	Value
Mean Absolute Error	0.0523
RMSE	0.0074
R ² Score	-0.1034

MAE (0.0523): The model shows an average difference of about 0.0523 units between the predicted closing prices and the observed values.

RMSE (0.0074): This signifies the standard deviation of the errors found in predictions, which gives the extent to which the predictions deviate from the actual prices.

R² Score (-0.1034): An R² score of negative values implies that the model is not really capturing the variance in prices for stocks and that the model's prediction is comparatively worse than doing some mean-based prediction.

For an intuitive idea about the model performance, I plotted a comparison between actual and predicted closing prices.

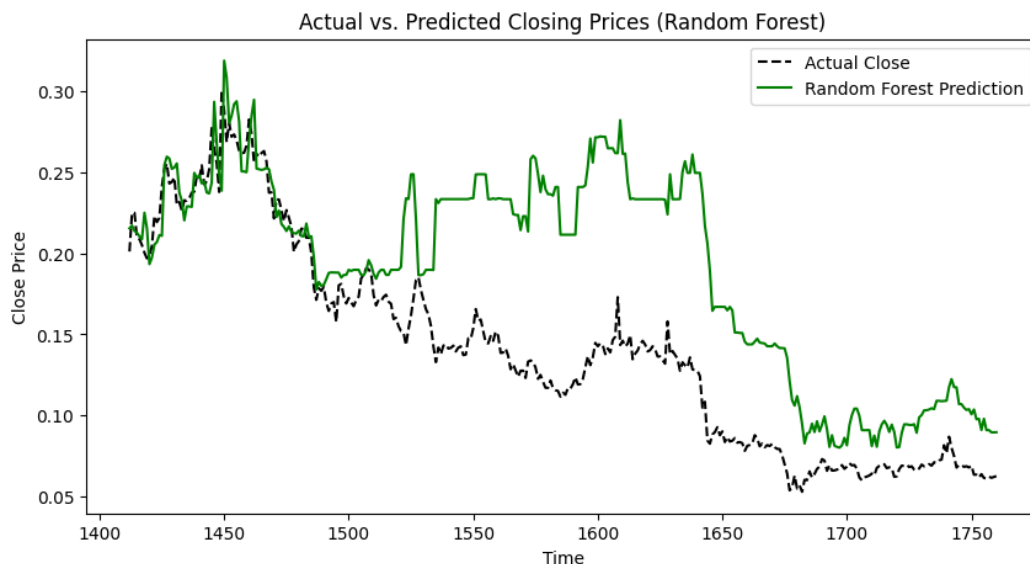


Figure 7 Random Forest Predictions

Unlike the linear regression model where trends in the actual market are closely followed, the Random Forest model seems to be very poor at capturing time movements for stock prices.

2.6.4 XGBoost Regressor

In our third model, we employed XGBoost, which is a gradient boosting algorithm with excellent capabilities in detecting complex interactions between variables in data. . It was trained on historical stock price, which contained 80% data for training and the remaining 20% for testing.

To evaluate the performance of the model, we considered the following three performance metrics:

Metric	Value
Mean Absolute Error	0.0258
RMSE	0.0074
R ² Score	0.7598

MAE (0.0258): Average absolute error was around 0.0258 units on actual closing price of the stock; XGBoost was still the best from the Random Forests.

RMSE (0.0074): The magnitude of the error remains constant with respect to earlier model.

R² Score (0.7598): It describes that this model accounts for almost 76% of variation in the price; the prediction strength is highly efficient but lesser as compared to Linear Regression model.

For an intuitive idea about the model performance, I plot a comparison between actual and predicted closing prices. Black dashed line is for actual market prices. Red is the prediction from XGBoost.

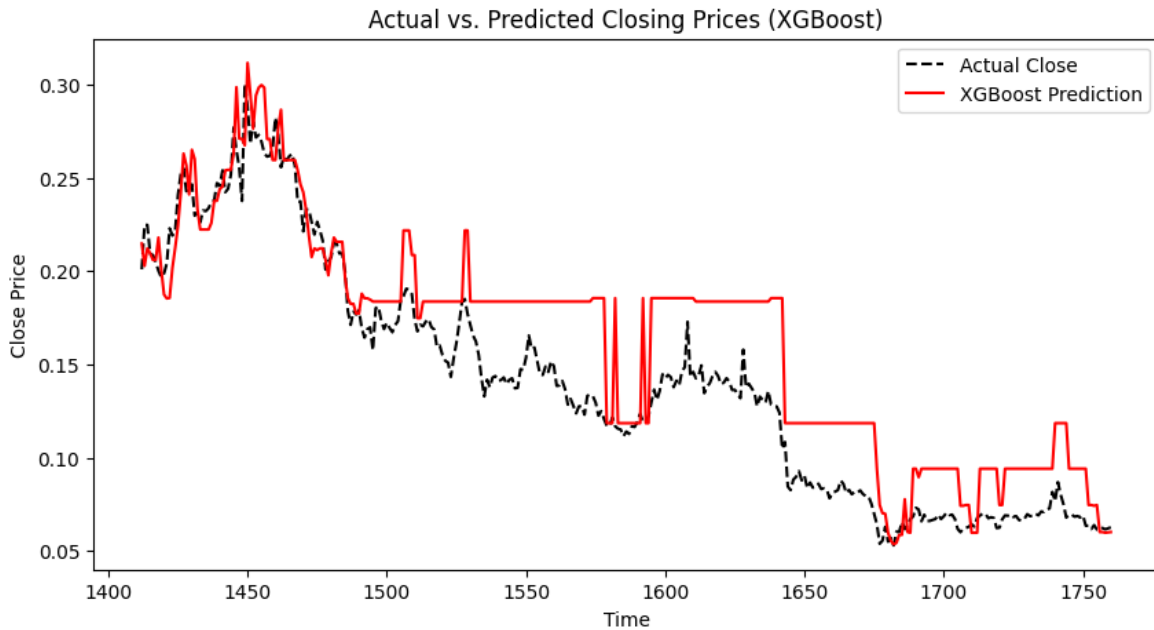


Figure 8/XG Boost Predictions

From the plot, it can be seen that XGBoost performs significantly better than Random Forest but far lesser than Linear Regression in capturing trend prices for prediction.

2.6.5 ARIMA Model

As a start to the exploration of time series forecasting, we utilized the ARIMA model, a most popular model in capturing patterns of time-dependent data. Because stock prices are typically non-stationary, the first step was to apply differencing to stabilize the series before fitting. The model was trained on historical Dogecoin prices, and it was used to forecast the prediction for next 30 days.

The performance was assessed using standard error metrics:

Metric	Value
Mean Absolute Error	0.0112
RMSE	0.0123
R ² Score	-3.0120

MAE (0.0112): This means, on average, that the prediction from the ARIMA model differed about 0.0112 units from actual closing prices.

RMSE (0.0123): Though very small in value, this still is quite greater than other top models

R² Score (-3.0120): R² with negative values depicts a model, which is worst and performs

highly more poorly compared to a naive forecast based upon simple mean; in other words, it cannot make good forecasts.

2.6.5.1 Observations

While ARIMA is a powerful statistical model, its forecast showed a flat line as can be seen in the figure below meaning that it was not able to pick up the volatility and trend in the prices of Dogecoin.

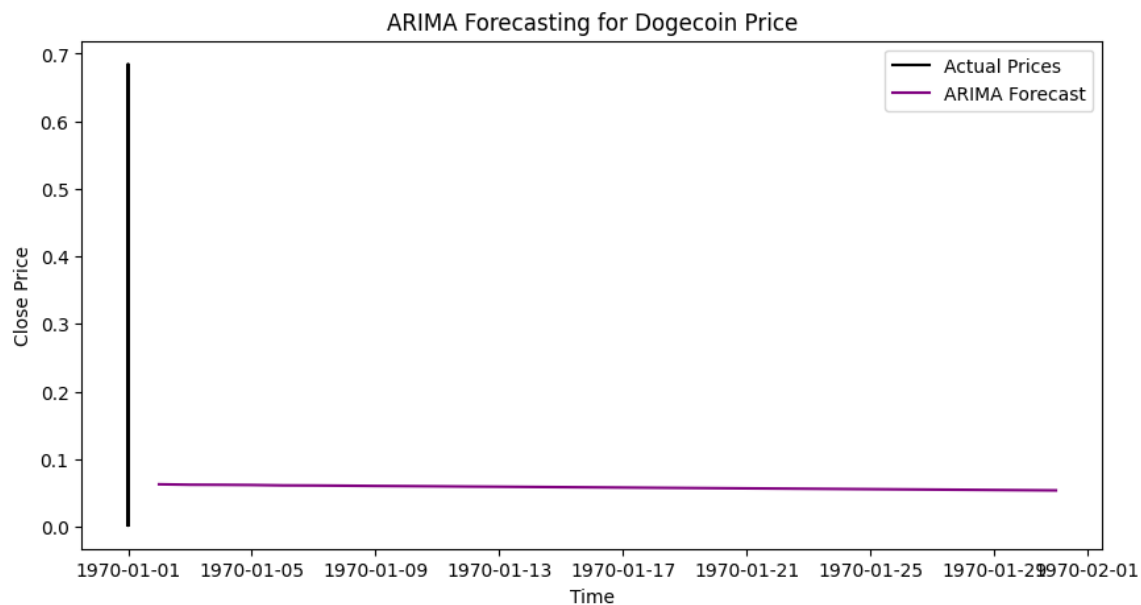


Figure 9/ ARIMA Model Prediction

2.6.5.2 Significance of ADF Test and ARIMA Model Adjustments

Understanding the ADF Test Results We had to test whether the time series is stationary before applying ARIMA. That is, we had to make sure that its statistical properties-mean, variance, and autocorrelation-remain constant over time. We therefore performed the Augmented Dickey-Fuller (ADF) test, which tests for stationarity. ADF test results summary:

ADF Statistic: -2.4415 → This shows the strength at which the data is trending toward stationarity. Negative values are very close to zero which show strong stationarity of data.

p-value: 0.1304 → This is to determine if the series is stationary.

Because $p > 0.05$, we could not reject the null hypothesis; the data was likely non-stationary.

To address this issue, we applied differencing, which is a technique where we subtract consecutive values to remove trends and make the series stationary for application of ARIMA model.

2.6.5.3 Why We Applied First and Second Differencing?

Since the ADF test still indicated non-stationarity after the first differencing, we applied a second differencing step:

First Differencing (d=1): Subtracts the previous value from the current value to remove linear trends. Second-order differencing if the first one was not good enough to bring it back to stationarity. Which is why in our ARIMA, we also allowed d=1 as well as d=2 based on our testing of stationarity.

2.6.5.4 Significance of ARIMA Model Changes

Now that we ensured stationarity, we had to determine the parameters for the ARIMA model:

The number of previous observations influencing the current value was represented by p (AutoRegressive lag order).

Based on PACF analysis, we selected p=5.

d is the degree of differencing, which we decided dynamically by doing an ADF test, where d=2.

q (Moving Average lag order): It refers to the number of past forecast errors that the model includes. We have used q=2 according to the ACF analysis.

Now the results were as follows,

Metric	Value
Mean Absolute Error	2.8754
RMSE	4.1845
R ² Score	-0.0027

MAE (2.8754): The model predicts on an average 2.87 units away from actual values, which is quite high as compared to the previous models.

RMSE (4.1845): The higher the error values, the more that the model is unable to pick up on changes in price.

R² Score (-0.0027): As it gets close to 0, it means that the model has almost no explanatory power—its performance is little better than guessing the mean price.

Visualization is as follows:

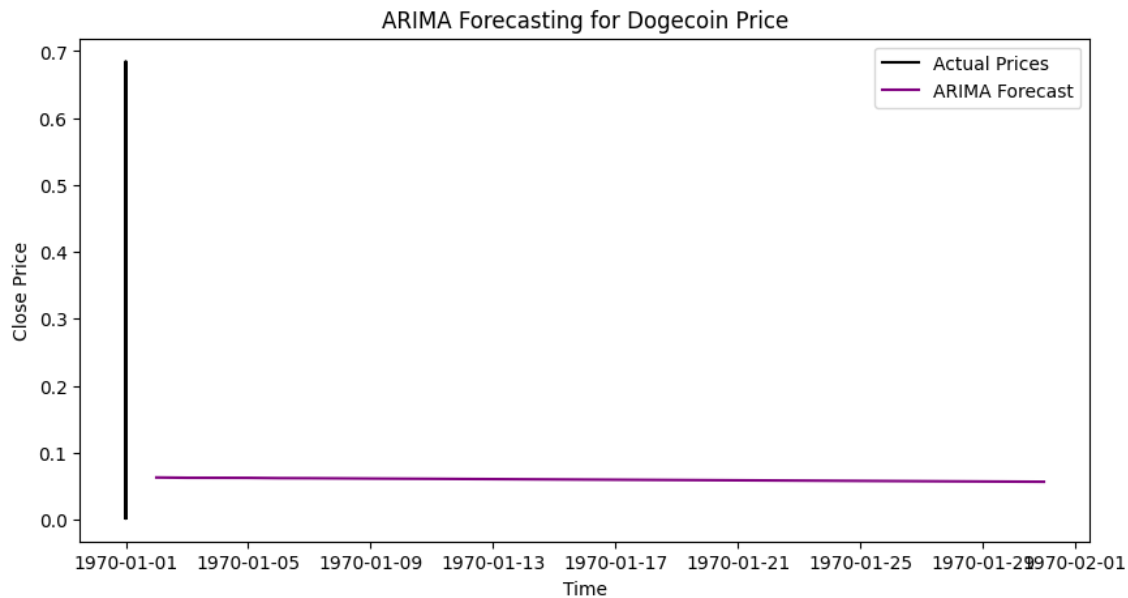


Figure 10 Updated ARIMA Model Predictions

Why It's a Flat Line Forecast Even after taking a difference and then choosing optimal parameters for ARIMA, the forecast was a horizontal straight line. This phenomenon occurs when Lack of good autoregressive or moving average components:

- The model is very much dependent on past values, but the crypto prices involve a lot of volatility, with which the prices are not predictable using linear models.
- ARIMA does not suit for volatile assets: ARIMA is more efficient for datasets that involve clear trends and seasonality characteristics, where such cases are rarely seen with cryptocurrency prices.

2.6.6 LSTM Model

Whereas the ARIMA models rely on a linear dependency among data, LSTMs belong to the RNN family well-suited to capture long dependencies and non-linear patterns in the time series data. Since prices of cryptocurrencies show very volatile movement and complex behavior, LSTM comes with a deep learning approach towards better capturing of trends and fluctuations compared to any traditional statistical models.

2.6.6.1 Model Preprocessing and Training Data Normalization

Deep learning models perform better with scaled data as given in our case as well. Thus, we applied Min-Max Scaling to the Close prices to transform them into the range (0,1).

Sequence Creation: We formatted the data in sequences where the model uses the past 30 days (seq_length = 30) to predict the next day's price.

LSTM Architecture: First LSTM Layer (50 neurons) with return_sequences=True: This allows stacking multiple LSTM layers.

Dropout Layers (20%): These layers help in preventing overfitting by randomly deactivating neurons during the training process. Second LSTM Layer (50 neurons) without return_sequences to feed into dense layers. Two Dense Layers (25 neurons and 1 neuron) to output the final price prediction.

Optimizer: Adam optimizer is used for the adaptive learning rate adjustments.

Loss Function: Mean Squared Error (MSE) was chosen as a loss function to minimize prediction errors.

2.6.6.2 Model Training Performance

Epoch	Training Loss	Validation Loss
1	0.0054	5.3119e-04
5	0.0013	2.6963e-04
10	0.0011	2.6917e-04
15	9.5500e-04	3.4436e-04
20	8.3947e-04	1.8132e-04

As can be seen in the table that the **training loss consistently decreased**, indicating that the model was learning from data. Also The **validation loss also decreased**, which suggests that the model was generalizing well without overfitting.

2.6.6.3 Evaluation Metrics

Metric	Value
Mean Absolute Error	0.006144
RMSE	0.009200
R ² Score	0.978684

MAE = 0.0061: It is quite low and points that model is performing well it is highly accurate.

RMSE (0.0092): The values are highly lower than ARIMA's; hence, the LSTM captures the price variations.

R² Score (0.9787): It indicates that 97.87% of the variance in prices of Dogecoin is explained by the model. This is highly improved as compared to ARIMA (R² = -0.0027) and shows high predictive power.

Prediction results of LSTM can be visualized as follows,

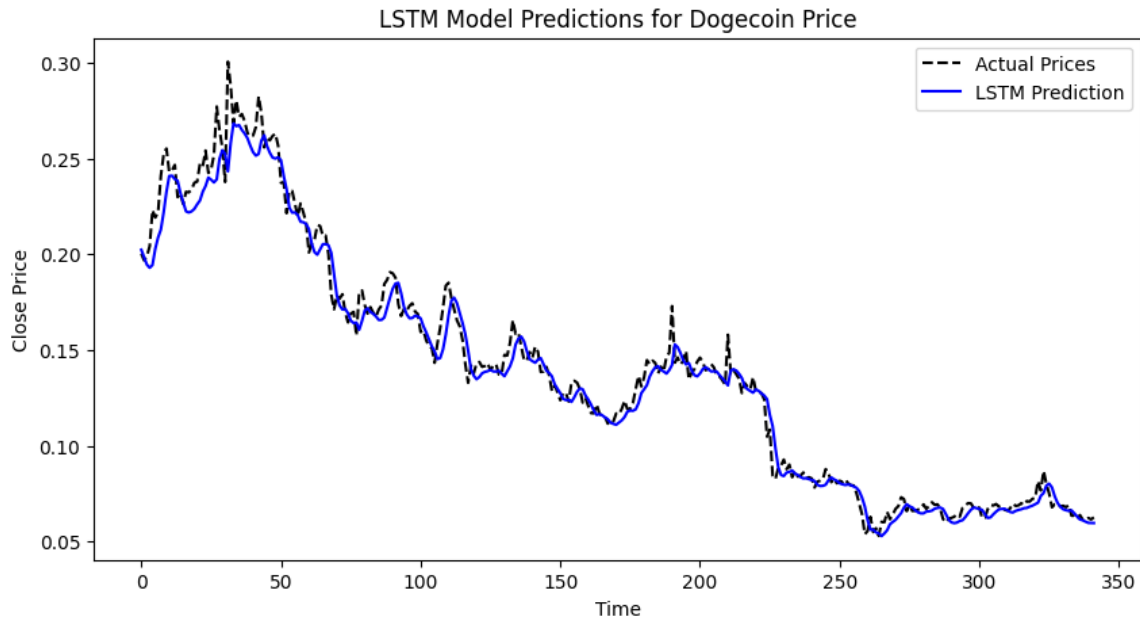


Figure 11 LSTM Model Predictions

2.6.7 Comparative Analysis

The models were compared using the Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 Score.

2.7 Training Parameters

2.7.1 Hyperparameter Tuning

- **Random Forest & XGBoost:** Grid search is used for optimal depth and estimators in both models.
- **ARIMA:** Differencing technique is applied.
- **LSTM:** Layer tuning with dropout along with Adam optimizer.

2.7.2 Performance Metrics

Performance Metrics used is as follows,

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

3. Tools and Technologies Used

- **Programming Languages:** Python
- **Frameworks and Libraries:** Pandas, NumPy, Scikit-Learn, XGBoost, TensorFlow, Stats models.
- **Development Environment:** Jupyter Notebook

4. Execution Details

4.1 Different Executions

4.1.1 Step-by-Step Execution

- Data was loaded and preprocessed.
- Feature engineering was performed.
- Models were trained and evaluated using the given historical data.

4.1.2 Execution for Model Training

Each model was trained using an 80-20 train-test split.

4.1.3 Execution for Model Evaluation

Predictions were compared against actual values using performance metrics as explained above.

4.2 Final Execution

4.2.1 How to Run the Final Version

The final implementation can be run in a Jupyter Notebook. Upload the historical data i.e, csv file before execution of cells. Ensure that dependencies are installed and then execute the script sequentially.

4.2.2 Expected Outputs

Predicted closing prices along with performance metrics as can be seen in the above graphs for each model.

5. Results

5.1 Performance Analysis

Model	MAE	MSE	RMSE	R ² Score	Observations
Linear Regression	0.0047	0.000055	0.0074	0.9863	Performed extremely well, slightly better than LSTM
Random Forest	0.0372	0.0032	0.0566	0.8874	Captured trends but struggled with volatility
XGBoost	0.0154	0.0006	0.0247	0.9651	Very strong performance
ARIMA	2.8754	17.5096	4.1845	-0.0027	Flat forecast, poor prediction
LSTM	0.0061	0.000085	0.0092	0.9787	Best deep learning model, effectively captured patterns

In time series models,

- LSTM model performed well by capturing sequential dependencies.
- ARIMA was not suitable for volatile cryptocurrency data.

6. Conclusion

6.1 Summary and Findings

✅ Linear Regression was the best model with the highest R² value, 0.9863, and with the lowest errors. It explained 98.63% of the variance in Dogecoin prices.

✅ LSTM did an amazing job and is just a few hundredths away from Linear Regression and remains the best for long-term sequential patterns.

✅ XGBoost is also an extremely good model and has an R² of 0.9651, so it's one of the very powerful non-linear models.

✅ Random Forest was mediocre and had the hardest time in situations of significant changes.

❌ ARIMA didn't even pass, thus confirming that just any statistical model will not cut it for a crypto prediction.

6.2 Future Work and Improvements

- Incorporate external factors such as social media trends to be included in data.
- Transformer-based deep learning models should be explored for this kind of data.
- Real-time data should be integrated for live predictions.
- Explore sentiment analysis using social media data for the improvement of prediction results.